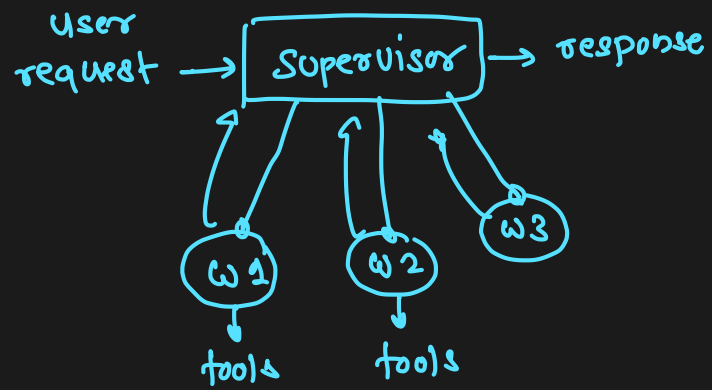


Supervisor

Introduction

→ group efforts

- The supervisor design pattern is an orchestration architecture for multi-agent systems
- It contains central coordinating agent—the supervisor—receives user requests, delegates subtasks to specialized worker agents, and aggregates their outputs into a final response
- Rather than having a single monolithic agent handle every aspect of a complex task, this pattern distributes work across focused specialists while maintaining centralized control over routing, sequencing, and result synthesis
- The supervisor does not perform specialist work itself when worker agents are available
- Its role is fundamentally about orchestration: understanding intent, breaking requests into subtasks, selecting the appropriate worker for each subtask, passing necessary context, waiting for results, and deciding whether additional delegation is needed before producing a final coordinated answer

Architecture: Structural Components

- A supervised agent team consists of three core elements:
- **Supervisor Agent:** The orchestrator that receives the user request, makes routing decisions, and maintains the overall task context
- **Worker Agents:** Specialized agents with focused responsibilities—such as querying databases, performing calculations, retrieving documents, or executing transactions—each with their own configured tools
- **Delegation Mechanism:** A tool-based handoff system that allows the supervisor to transfer control to a worker and receive results back
- In implementation frameworks like LangGraph, the supervisor creates handoff tools automatically for each worker agent (e.g., handoff_to_math, handoff_to_research)
- These tools encapsulate the transfer of task context to the appropriate specialist

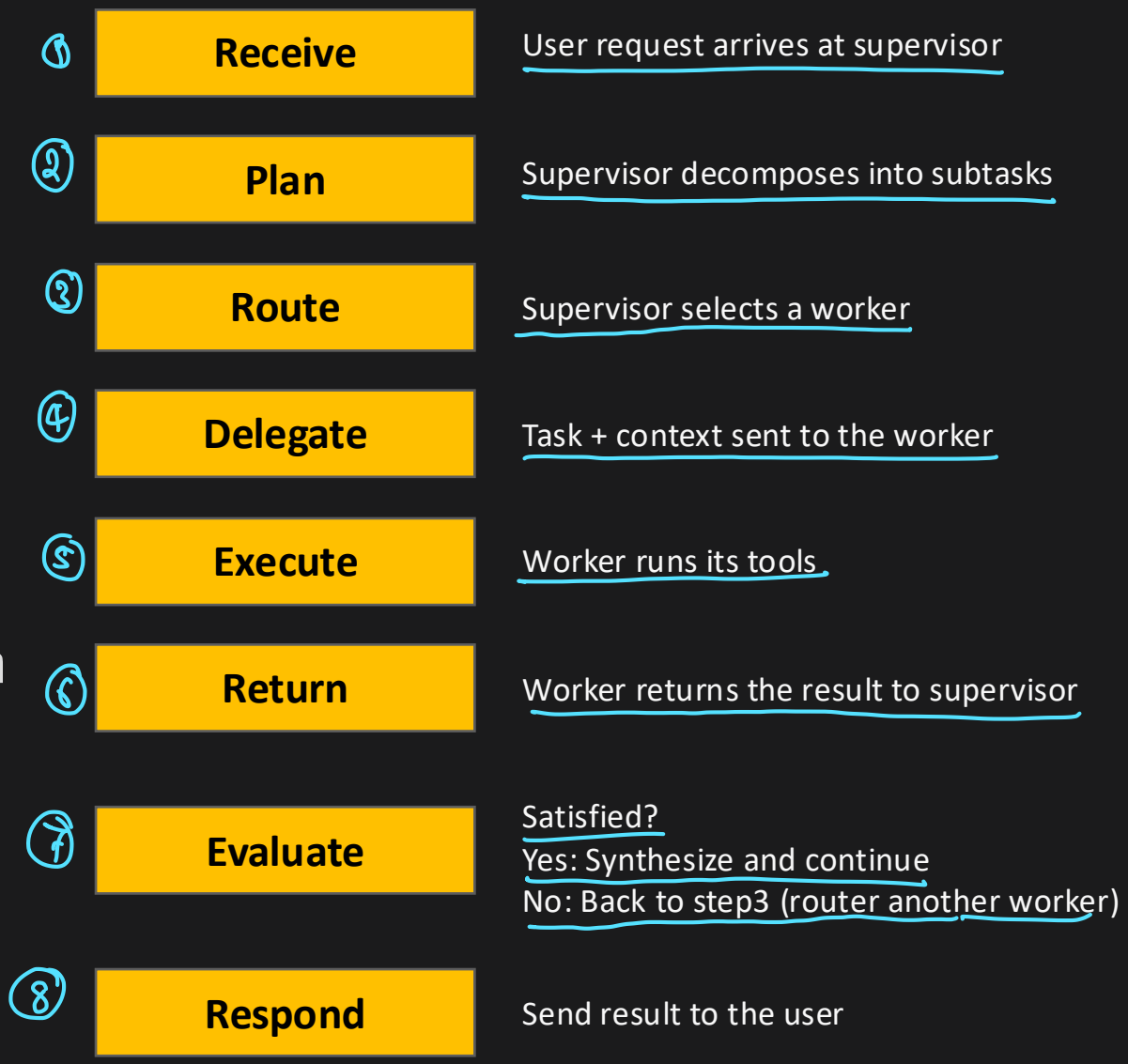
Component Roles

- Supervisor (Manager) → brain
  - Understand the user's intent
  - Break the task into subtasks → routing
  - Decide which worker handles each subtask
  - Pass the right context to each worker
  - Evaluate whether results are sufficient
  - Re-route or retry when needed
  - Combine outputs into a final answer
  - Does NOT
    - Execute domain-specific work itself (worker agent)
    - Hold specialist tools
    - Override worker expertise

- Worker (Specialist)
  - Handle one narrow domain
  - Own a focused set of tools
  - Return clean, structured results
  - Does NOT
    - Route to other workers (unless explicitly allowed)
    - Synthesize final answers
    - Handle tasks outside their domain

Sequential Task Processing

- The subtask process is sequential: a worker subtask must complete before the supervisor delegates the next one
- The supervisor waits for the worker's result, adds it to its context, and then decides whether to call another worker, request more information, explain a failure, or return the final response
- Worker agents typically use separate conversation threads, which means the supervisor must include sufficient context in each delegated request
- This isolation prevents cross-contamination of context but requires deliberate context management by the supervisor



Design Considerations

- **Routing and Delegation Strategy**
  - The supervisor's routing rules should be concrete and predictable
  - Effective supervisor instructions specify which agent or tool handles which type of request
  - For example:
    - Use the SQL agent for structured data questions, the HTTP agent for external API lookups, and the RAG agent for document-based questions
    - This explicitness reduces ambiguity in routing decisions and improves system reliability
- **Context Passing and Isolation**
  - A critical design decision concerns how much context to pass to worker agents
  - Frameworks support multiple modes
    - **Full history:** The worker receives the complete conversation history from the supervisor's context
    - **Last message only:** The worker sees only the immediate task assignment
    - **Fresh context:** Each worker receives only the specific task, not the full conversation history
- **Iteration Limits and Error Handling**
  - Supervisor implementations typically include maximum iteration limits to prevent infinite delegation loops
  - Worker retries can also be configured to handle transient failures gracefully

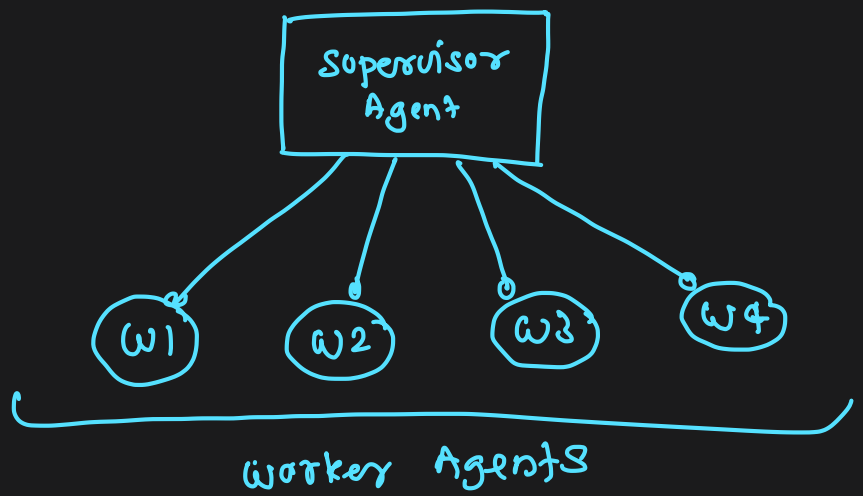
Advanced Patterns

- **Human-in-the-Loop Integration**
  - Supervisor systems can incorporate approval gates for sensitive operations
  - When a worker attempts a high-stakes action—such as transferring funds or allocating budget—an interrupt is triggered and the system pauses for user approval
  - The workflow resumes from the exact point of interruption after the user responds
- **Nested and Hierarchical Supervisors**
  - Multi-level hierarchies allow supervisors to manage other supervisors : supervisor of supervisors
  - For example, a top-level supervisor might coordinate a research team supervisor and a writing team supervisor, each of which manages their own worker agents
  - This enables complex organizational structures that mirror human team hierarchies
- **Plan-Execute-Replan Sub-Agents**
  - A supervisor can delegate to a complex sub-agent that itself follows a Plan-Execute-Replan pattern
  - Creating a multi-step plan, executing each step, and adjusting the plan based on intermediate results
  - This allows the supervisor to delegate entire complex workflows while maintaining high-level oversight

Topology Options

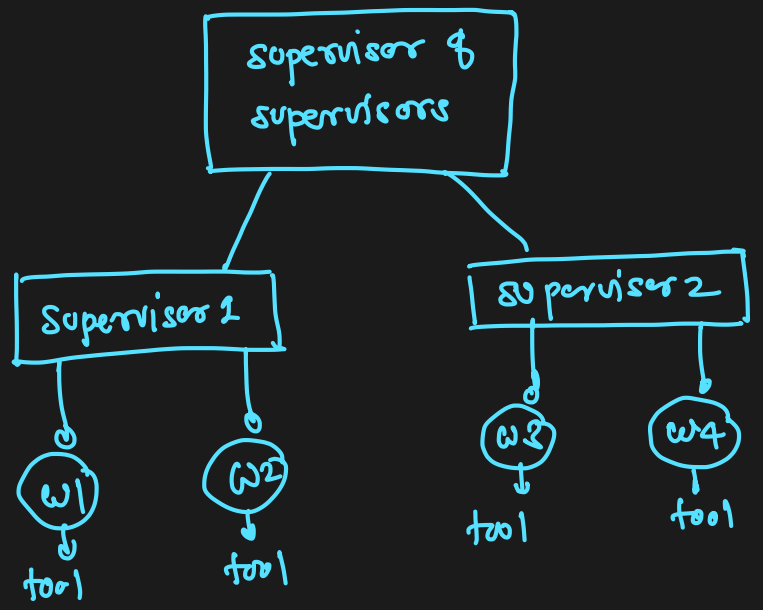
Flat / Star Topology

- One supervisor, N workers, all communication flows through the center
- Workers never talk to each other
- **Pros**
  - Simplest to build and reason about
  - Centralized logging and observability
  - Easy to add/remove workers
- **Cons**
  - Supervisor is a bottleneck (all traffic passes through it)
  - No parallelism unless the supervisor explicitly fans out
  - Single point of failure
- **Best for:** Small teams (2–5 workers), demos, MVPs, well-defined domains



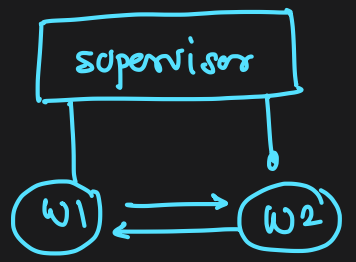
Hierarchical (Supervisor of Supervisors)

- Top supervisor manages sub-supervisors; each sub-supervisor manages its own workers
- Two or more levels of coordination
- **Pros:**
  - Scales to very large teams (dozens of agents)
  - Mirrors real organizational structures
  - Each sub-team has focused responsibilities
  - Top supervisor sees a simplified view (teams, not individuals)
- **Cons:**
  - Higher latency (multiple coordination hops)
  - More complex to debug
  - Risk of the top supervisor being too abstract
  - Sub-supervisors add token cost
- **Best for:** Enterprise-scale systems with clearly separated business domains (e.g., sales team, support team, engineering team)



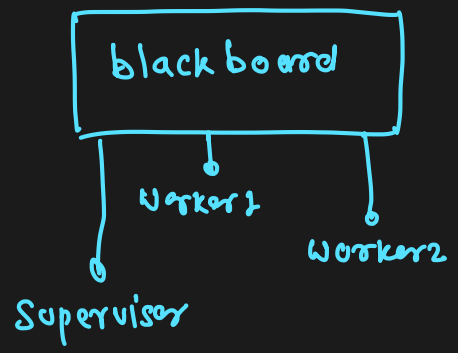
Supervisor + Mesh (Hybrid)

- Supervisor handles initial routing and final synthesis, but workers can hand off to each other directly for specific tasks
- e.g., a research worker passing data to an analysis worker without supervisor involvement
- **Pros:**
  - Faster for worker-to-worker handoffs
  - Reduces supervisor load
  - Keeps centralized control for high-level decisions
- **Cons:**
  - Harder to trace (traffic doesn't always flow through the center)
  - Workers need to know about each other
  - Risk of bypassing supervisor guardrails
- **Best for:** Pipelines where workers have natural handoff relationships (retrieve → analyze → write)

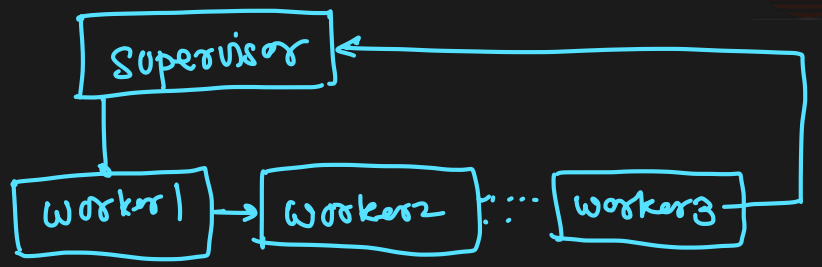


Blackboard (Shared State)

- All agents read from and write to a shared data store
- The supervisor publishes tasks; workers pick them up and post results
- **Pros:**
  - Highly decoupled — workers don't need to know each other
  - Natural fit for async / long-running workflows
  - Easy to add workers (they just subscribe to relevant events)
- **Cons:**
  - Race conditions and ordering issues
  - Harder to enforce strict sequencing
  - Debugging requires tracing the shared state, not just messages
- **Best for:** Event-driven systems, long-running workflows, agents that react to conditions rather than requests



Pipeline (Linear Supervisor)



- Workers are chained in a fixed order
- Supervisor enforces the sequence and passes outputs forward
- **Pros:**
  - Deterministic and reproducible
  - Easy to test and debug
  - Low coordination overhead
- **Cons:**
  - No dynamic routing
  - Can't skip or reorder steps
  - Fails if a step produces bad output
- **Best for:** Well-defined workflows (ETL, document processing, multi-stage analysis).

Use of Pattern



- When to use
  - Workflows span distinct business or technical domains requiring different expertise
  - Requests require different agents, tools, or data sources in varying orders
  - The team needs a single entry point for routing and coordination
  - The order of execution cannot be predetermined and must be decided at runtime based on user intent and intermediate results
- When to avoid
  - The pattern is less suitable when a single agent with tool access can handle the full scope of tasks, or when strict deterministic workflows with no runtime routing decisions are required.

Best Practices



- **Keep supervisor instructions concrete:** Specify routing rules explicitly so decisions are predictable and debuggable
- **Define clear worker specialties:** Each worker agent should have a focused responsibility with its own configured tools
- **Manage context deliberately:** Choose isolation levels based on continuity needs versus privacy and prompt-injection risk
- **Set iteration limits:** Prevent infinite loops with reasonable maximum iteration counts
- **Use meaningful names:** Descriptive agent names aid debugging and trace interpretation
- **Add guardrails:** Consider PII filtering, content moderation, and prompt-injection detection before and after the supervisor

Pros and Cons



- Pros
  - Centralized Control and Observability
  - Clear Separation of Concerns
  - Dynamic Routing at Runtime
  - Easier to Add or Remove Specialists
  - Natural Fit for Human-in-the-Loop
  - Context Isolation
  - Supports Hierarchies
- Cons
  - Single Point of Failure
  - Supervisor Becomes a Bottleneck
  - High Token Cost
  - Requires Reliable Structured Output
  - Prompt Engineering Burden
  - Hard to Make Deterministic

Design Approaches

Routing Logic



- **LLM-as-Router (most common)**
  - Give the supervisor a prompt listing each worker's name, description, and capabilities
  - The supervisor's LLM reads the user request and emits a tool call to the matching worker
  - Works well when routing rules are semantic and fuzzy (e.g., "billing question" vs "technical question")
- **Rule-Based / Deterministic Router**
  - Use keyword matching, regex, or intent classifiers to route requests
  - Faster and cheaper than LLM routing, but brittle for ambiguous queries
  - Good for high-volume, well-defined domains (e.g., "refund" → billing agent)
- **Hybrid Router**
  - First pass: cheap rules handle obvious cases
  - Fallback: LLM decides for ambiguous cases
  - Reduces cost and latency while keeping flexibility
- **Capability-Registry Router**
  - Each worker registers a set of "skills" or "tool signatures."
  - The supervisor matches the task to skills rather than to named agents
  - Makes adding/removing workers easier—no prompt changes needed

Delegation Mechanism

- **Tool-Call Handoff**
  - Each worker is exposed as a callable tool to the supervisor
  - Supervisor calls tool and receives a result
  - Works with any LLM that supports tool calling
- **Message-Passing / Shared State**
  - All agents read/write to a shared blackboard or state object
  - Supervisor writes a task; the target worker picks it up
  - Useful for async or parallel execution
- **Explicit Control Transfer (Handoff)**
  - Supervisor fully yields control to the worker, which then returns control when done
  - Common in graph-based frameworks (LangGraph nodes and edges)
  - Enables worker-to-worker handoffs without supervisor in the loop
- **Queue-Based Delegation**
  - Supervisor pushes tasks onto per-worker queues
  - Workers pull, process, and push results back
  - Good for scaling, retries, and observability

Context Management



- **Full History Sharing**
  - Worker sees the entire conversation
  - Pros: maximum continuity, worker understands nuance
  - Cons: token cost, prompt-injection risk, context bloat
- **Summary + Task Only**
  - Supervisor summarizes prior context and passes only the summary + current task
  - Balances continuity with cost and isolation
- **Isolated Task Context**
  - Worker sees only its assigned task, nothingelse
  - Pros: clean, secure, parallelizable
  - Cons: worker may lack needed background
- **Scoped Memory**
  - Each worker has its own memory store; supervisor has its own
  - Shared facts go in a global store; private reasoning stays local
- **Structured Handoff Object**
  - Pass a typed object: {task, constraints, prior_results, expected_output_format}
  - Forces the supervisor to be explicit; makes workers easier to test

Task Decomposition



- **Static Playbooks**
  - Predefined workflows (e.g., "for a refund request: verify → check policy → issue refund")
  - Predictable, testable, but inflexible
- **Dynamic Planning (Plan-and-Execute)**
  - Supervisor generates a plan at runtime, then executes step by step
  - Handles novel requests; requires good planning prompts
- **ReAct-Style Interleaving**
  - Supervisor thinks → acts (delegates) → observes result → thinks again
  - Simple to implement; good for exploratory tasks
- **Recursive Decomposition**
  - Supervisor splits a task, then delegates sub-splits to sub-supervisors
  - Enables hierarchies for very complex workflows
- **Dependency Graph**
  - Supervisor builds a DAG of subtasks with dependencies
  - Independent branches run in parallel
  - Good for research + analysis + writing pipelines

Result Aggregation



- **Sequential Chaining**
  - Worker A's output becomes Worker B's input
  - Good for pipelines (retrieve → analyze → write)
- **Merge & Summarize**
  - Workers run in parallel; supervisor summarizes/merges results
  - Good for multi-perspective tasks (research + critique + fact-check)
- **Voting / Consensus**
  - Multiple workers answer the same question; supervisor picks the majority or best
  - Useful for high-stakes decisions
- **Conflict Resolution**
  - When workers disagree, supervisor asks a tie-breaker worker or applies rules
  - Explicit "critic" agent can adjudicate
- **Format Normalization**
  - Supervisor forces each worker to return a typed schema (JSON) for easy merging